

CS1632: Test Plans and TM

Wonsun Ahn

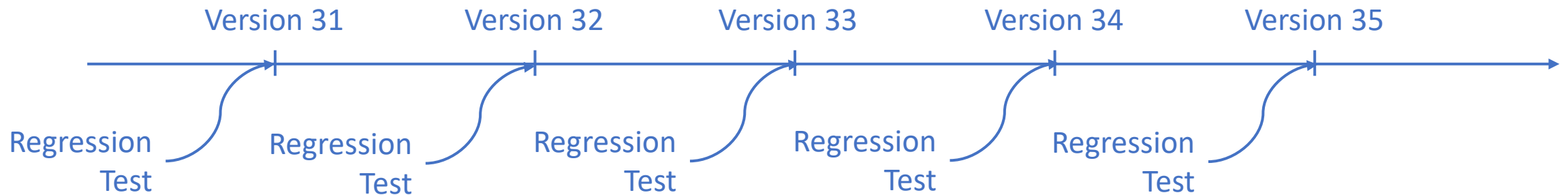
Test Plans

What is a Test Plan?

- Test Plan: A document laying out a plan for testing a software system
- Why do we need a **plan**?
 - Goal of testing is to minimize risk of defects given a time/cost budget
 - Careful planning is needed to maximize test coverage
- Why do we need to **document** the plan?
 - Allows project managers to estimate test coverage and manage risk
 - Allows quality engineers to reliably repeat the same tests over and over again
 - **Repeatability** of tests is particularly important for *regression tests*

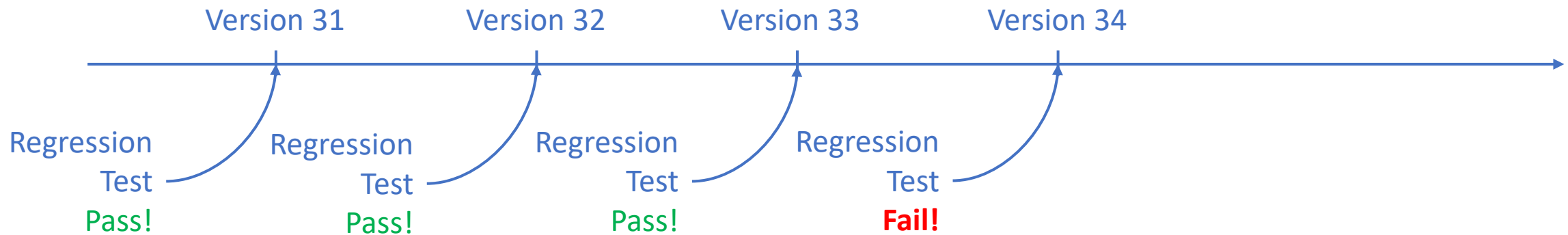
Regression Tests prevent SW from regressing

- *Regression*: A failure of a previously working feature
 - Can be caused by (seemingly) unrelated enhancements or defect fixes
 - Why? Because code fixes often have non-local effects
 - Regression test must test modified feature but also all other features
- For timely regression detection, regression test is run on each code update



Repeatable tests can pinpoint defective version

- Suppose a regression test fails on a code update:



- We can pinpoint where the defect crept in, at Version 34.
- Why? Because we are confident that we are repeating the same tests!
- Now, if we ran different tests each time, would we know?

How formal should the plan be documented?

- As formal or informal as necessary!
- Think about what you are testing
 - How critical is the software that you are testing?
 - How many times is the test plan going to be used?

What are you testing?

- Throw-away script?
- Development tool?
- Internal website?
- Enterprise software?
- Commercial software?
- Operating system?
- Avionics software?

Testing is context-dependent

- How you test
- How much you test
- What tools you use
- What documentation you provide
- ...All vary based on software context.

Test Cases

Test Plans and Test Cases

- *Test plan*: a carefully constructed plan for rigorously testing software
 - Consists of a list of test cases that provide test coverage for requirements
- *Test case*: a test scenario with precise steps on how to perform it
 - Describes what is to be tested and what steps to perform
 - Describes expected behavior after the steps are performed

Test Case main body consists of ...

- *Preconditions*: State of system before execution steps. E.g.,
 - Packages X and Y are installed on the system
 - Configuration file X contains entry Y
 - Database has table X set up populated with Y entry
- *Execution Steps*: Steps to perform test
- *Postconditions*: **Expected** state after execution steps
 - Derived from **requirements** based on preconditions + execution steps

Test Case header identifies and describes it

- *Test Case*: A short description of what is being tested
- *Identifier*: A way to identify the test case
 - Could be numerical, e.g. TEST_001
 - Or a descriptive label, e.g. TEST_INVALID-PASSWORD
 - We will use the former for this class

In full, a test case contains the following items

- Test Case
- Identifier
- Preconditions
- Execution Steps
- Postconditions

Ref. “Software and systems engineering — Software testing — Part 3: Test documentation”:
<https://ieeexplore-ieee-org.pitt.idm.oclc.org/document/9591577>

Example Test Case (Using Sphinx-Needs)

.. test:: Passing in the argument 34
 :id: TEST_002

Preconditions:

- The potus.jar file is in current folder.
- Java 11 is installed on the machine.

Execution Steps:

1. Open a shell and cd into the current folder.
2. Run: "java -jar potus.jar 34"

Postconditions:

- The system prints "34 is too young to be President of the United States".

Test Run – Actual execution

- *Test run*: Actual execution of a test case
 - Typically, a test case will undergo thousands of test runs in its lifetime
 - Subsets of test cases may be chosen to run from the entire test suite
- The purpose of a test run is to obtain *observed behavior*
 - Passes or fails after comparing *observed behavior* with *postcondition*

Status after Test Run

- Possible Statuses
 - PASSED: Completed with expected result
 - FAILED: Completed but unexpected result
 - PAUSED: Test paused in middle of execution
 - RUNNING: Test in the middle of execution
 - BLOCKED: Did not complete because precondition not fulfilled
 - ERROR: Problem with running test itself
- During test run, tester manually (or automatically) executes each test case and sets the status for each
- A FAILED status signals a defect that needs to be reported.

Creating Good Test Cases

A good test case is two things

- A good test case verifies **requirements faithfully**
 - No over-specification of **Postconditions**
 - If postconditions check fails, it is always due to a defect
 - No under-specification of **Postconditions**
 - If postconditions check passes, no defect for tested scenario
- A good test case is **repeatable**
 - **Preconditions + Execution Steps** are complete and unambiguous
 - Observed behavior is consistent regardless of who runs the test case

Pitfall 1: Over-specification

Using Observed Behavior for Postcondition

- Never use screenshots (or copy-and-paste) of output even if correct
 - Observed behavior contains aspects that are not part of required behavior
- Consider the following scenario:
 - Requirement: The sum of the 2 arguments is displayed.
 - Execution Step: Run “sum 1 2” in the shell.
 - Postcondition: **Result is: 3** is displayed.
 - Observed behavior: **Value is: 3** is displayed.
- The test case would fail, but is this a defect? No!
- Postconditions should be derived from **requirements faithfully**
 - Correct postcondition: The value 3 is displayed.

Pitfall 2: Under-specification

Using Execution Steps for Verification

- Never use Execution Steps to verify correctness
 - Only Postconditions are able to specify correct behavior
 - Execution Steps only describe steps to produce observed behavior
- What happens when an Execution Step does not match UI exactly?
 - E.g. Execution Step says button is at the top of page, but UI has it on the bottom
 - Often, testers will flexibly adjust the step to what they see on the screen
 - If step cannot be performed, the test result is **ERROR** not FAILED!
 - End result is not a defect report, but execution steps getting updated

Pitfall 3: Unrepeatability

Imprecise Preconditions / Execution Steps

- Incomplete preconditions (OS / DB / Filesystem)
 - E.g. The Linux version is unspecified, and result differs depending on version
 - E.g. A configuration file that impacts result is not specified
- Imprecise execution steps
 - E.g. “Open new browser” → Multiple ways: Ctrl+N, Menu, Double click
- Results in **unrepeatable** tests

Pitfall 4: Unrepeatability

Not Deriving Actual Behavior for Postconditions

- Consider the following scenario:
 - Requirement: System shall display Fibonacci sequence following argument.
 - Execution Step: Run “fib 5” in the shell.
 - Postcondition: Fibonacci sequence following argument is displayed.
 - Observed behavior: “8” is printed.
- Would the average tester be able to tell whether this is defective or not?
→ Leads to **unrepeatable** result depending on who does the testing.
- Often, expected behavior needs a derivation step from postconditions
 - May need knowledge of software that average tester does not have
 - Correct postcondition: The value 8 is displayed.

Pitfall 5: Unrepeatability

A Test that is Dependent on Another Test

- Test case shouldn't depend on the result of a previous test case
 - E.g. Should not depend on database entries inserted by previous test case
- Results in **unrepeatable** tests
 - If the previous test case fails, this test case will be impacted
 - Test cases may be run selectively, and previous case may not be selected to run
 - Test cases may execute out of order, causing previous case to execute later
(Often, test cases are executed in parallel to save testing time)
- Writing complete set of **Preconditions** prevents this from happening
 - Forces setup of all dependencies for test case prior to running it

Pitfall 6: Unrepeatability

A Test Case that merges Multiple Scenarios

- Example of a merged test case:
 - Execution Steps:
 1. Call `sqrt(4)`
 2. Call `sqrt(9)`
 3. Call `sqrt(16)`
 - Postconditions:
 - Results of first, second, and third calls are 2, 3, 4, respectively.
- Telltale sign is that postconditions check conditions mid-execution
 - Unclear when the expected behavior is to be checked → unrepeatable
 - On failure, unclear which test scenario failed (4? 9? or 16?)

Traceability Matrix

Traceability: Ability to trace requirements to test cases (and vice versa)

- Forward Traceability
 - Ability to trace **requirement** → **test cases**
 - Given a requirement, be able to list all test cases that test it
 - Can detect when requirements have insufficient test coverage
- Backward Traceability
 - Ability to trace **test case** → **requirements**
 - Given a test case, be able to list all requirements that are tested
 - Can detect “orphaned” test cases not testing any requirements
→ “Orphaned” tests need to be resolved or they may result in spurious code
- Can ensure requirements, and only requirements, are implemented

Traceability Matrix ensures traceability

- **Traceability Matrix:**
Table describing relationship between requirements and test cases
- Why is it a “matrix”?
 - One test case may test multiple requirements
 - One requirement may be tested by multiple test cases
 - It's a many-to-many relationship, hence the matrix

Good Forward Traceability Matrix Example

REQ1: TEST_CASE_1, TEST_CASE_2

REQ2: TEST_CASE_1, TEST_CASE_3

REQ3: TEST_CASE_1

REQ4: TEST_CASE_2

REQ5: TEST_CASE_4

- Mapping requirements → test cases
- All requirements have at least one test case testing that requirement

Bad Forward Traceability Matrix Example

REQ1: TEST_CASE_1, TEST_CASE_2

REQ2:

REQ3: TEST_CASE_1

REQ4: TEST_CASE_2

REQ5: TEST_CASE_4

- *No test case is testing requirement 2. Lack of test coverage!*

Good Backward Traceability Matrix Example

TEST_CASE_1: REQ1, REQ2, REQ3

TEST_CASE_2: REQ1, REQ4

TEST_CASE_3: REQ2

TEST_CASE_4: REQ5

- Mapping test cases → requirements
- All test cases have at least one requirement it is testing

Bad Backward Traceability Matrix Example

TEST_CASE_1: REQ1, REQ2, REQ3

TEST_CASE_2: REQ1, REQ4

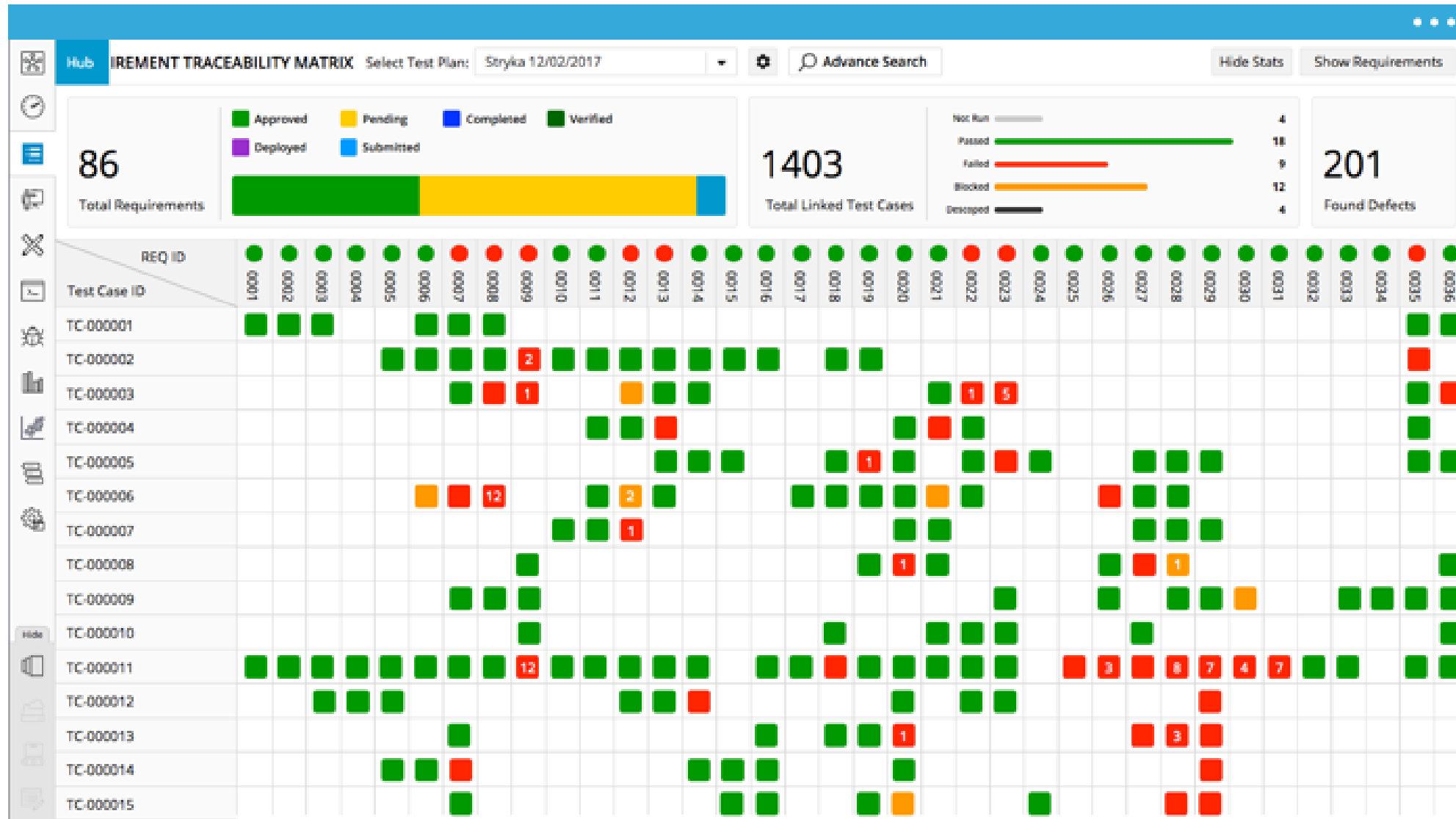
TEST_CASE_3: REQ2

TEST_CASE_4: REQ5

TEST_CASE_5:

- *Test case 5 not checking any requirement (orphaned test case).*

A Bi-Directional Traceability Matrix



Reference:
reportportal.io

Now Please Read Textbook Chapters 6 and 8

- (Optional) If you are interested in further reading:

Software and systems engineering — Software testing — Part 3: Test documentation (ISO/IEC/IEEE 29119-3)

<https://ieeexplore-ieee-org.pitt.idm.oclc.org/document/9591577>